Robust & Fair Federated GNN for Crime Prediction

A research extension of FedCrime — full technical report

What we built, how we solved each problem, every formula, and the real results on the Los Angeles and Chicago datasets.

Base paper: Bhumika, Lalanda, Vega, Das. *FedCrime*, Neurocomputing 679 (2026) 133217.

1. Overview — what this project is

We started from the **FedCrime** paper, which predicts city crime using *federated learning* (each area keeps its data private and only shares a model) and a *Zero-Inflated Negative Binomial* (ZINB) loss to handle the huge number of no-crime days (zero-inflation). We reproduced it, then extended it with three new problems the paper does not solve: **spatial dependency** (a graph neural network), **robustness** to malicious clients, and **fairness** across rich/poor neighbourhoods — plus **week-ahead** prediction and **uncertainty**.

> **Key idea:** The core research insight (our gap): in crime data the honest poor (sparse/"tail") neighbourhoods look just like malicious attackers, so standard defenses either let attacks through or unfairly silence poor regions. This is the **sparsity–robustness–fairness trilemma**, which no prior work addresses.

Everything runs on the real **Los Angeles (113 areas, 2018)** and **Chicago (77 areas, 2015)** data.

2. What we built (the components)

Component	What it does	Paper / formula
Data pipeline	Raw crimes -> daily yes/no windows; time split; Head/Mid/Tail; region graph	Sec. 5
TCN-SD model	Reads recent days, outputs zero-inflation, mean, dispersion	Eq. 3-8
ZINB loss	Handles the many zeros + over-dispersion	Eq. 9-11
Federated averaging	Combines client models on the server (FedAvg)	Eq. 12
GNN (plain/gated/attention)	Regions share info through a graph; gated fixes over-smoothing	Sec. 6
Attacks	Label-flip, scaling, and the novel sparsity-camouflaged attack	Sec. 7
Defenses	Trimmed-mean, Multi-Krum, and our density-aware "vouch"	Sec. 8
Fairness + uncertainty	Head/Mid/Tail gap; week-ahead; MC-dropout confidence	Sec. 9

3. The base model and its formulas

3.1 Temporal Convolutional Network (TCN) — Eq. 3-5

The TCN reads the last few days for each region using dilated causal convolutions (it only looks backward in time, and skips steps so it can see far back cheaply).

```
Eq 3:  h(l,t)   = ReLU( W1 . X(t - d*k) + b1 )      k = 0..K-1
Eq 4:  h(l+1,t) = ReLU( W2 . h(l, t - d*k) + b2 )
Eq 5:  h_tcn(t) = h(l+1,t) + Residual( X(t - d*k) )   (skip connection)
```

W = learnable filters, d = dilation (grows 1,2,4 with depth), ReLU keeps positives. The skip connection (Eq 5) keeps gradients flowing.

3.2 Statistical head — the ZINB parameters — Eq. 6-8

From the TCN summary h, three small layers output the three ingredients of a ZINB distribution, each squeezed into the right range.

```
Eq 6:  pi  = sigmoid ( W_pi . h + b_pi )     zero-inflation probability (0..1)
Eq 7:  mu  = exp     ( W_mu . h + b_mu )     mean count            (> 0)
Eq 8:  phi = softplus( W_phi. h + b_phi)     dispersion / spread   (> 0)
```

3.3 The sparsity-aware ZINB loss — Eq. 9-11

```
Eq 9:   L_zero = 1{Y=0} * log( pi + eps )
Eq 10:  L_NB   = 1{Y=1} * [ logGamma(Y+phi) - logGamma(phi) - logGamma(Y+1)
                            + phi*( log phi - log(phi+mu) )
                            +  Y *( log mu  - log(phi+mu) ) ]
Eq 11:  J = - E[ L_zero + L_NB ]        (minimise this)
```

The zero term rewards confident zeros (quiet areas); the NB term shapes the counts where crime occurs. Sparse areas produce small, calm updates; busy areas produce sharp, informative ones — so the loss itself balances them.

3.4 Federated averaging — Eq. 12

```
Eq 12:  W = (1/|R|) * sum over clients i of  w_i
```

The server just averages the client models. FedCrime's point: get the loss right and simple averaging is enough.

3.5 Fix for real data: class-weighted classification head

On the real data (68% zeros) the model collapsed to predicting “no crime”. We added a classifier head trained with class-weighted cross-entropy, keeping ZINB as an auxiliary term:

```
loss = BCE_with_pos_weight( logit, Y )  +  0.3 * ZINB_loss
pos_weight_c = (#neg_c / #pos_c)   (rare crimes weighted up: theft x1 ... sex-offenses x6.8)
prediction: crime present  <=>  sigmoid(logit) > 0.5
```

4. The GNN extension and the over-smoothing problem

4.1 The region graph

The real data has no graph, so we build one: connect each region to its top-k most similar regions (by correlation of their daily crime), then symmetrically normalise.

```
corr = correlation( region daily-crime series )
A[i,j] = 1  if j is among region i's top-k most similar   (+ self loops)
A_hat  = D^-1/2 (A) D^-1/2       (normalised adjacency)
```

4.2 Plain graph convolution — and why it HURT accuracy

```
Plain GCN:   H' = ReLU( A_hat . H . W )
```

This averages each region with its neighbours. But busy (Head) and quiet (Tail) regions are very different, so averaging **blurs the signal** and lowers accuracy. This well-known effect is called **over-smoothing**. On real data the plain GNN scored BELOW the no-graph model (see results).

4.3 The fix 1 — Gated graph convolution

```
g  = sigmoid( W_g . H )                 # learned gate in 0..1, per region
H' = g * (A_hat . H . W)  +  (1 - g) * H  # blend neighbours vs own signal
```

Each region **learns** how much to trust neighbours. Busy regions turn the graph down (keep their own strong signal); quiet regions turn it up (borrow from neighbours). This recovers the lost accuracy.

4.4 The fix 2 — Attention graph convolution

```
alpha_ij = softmax_j( (W h_i) . (W h_j) / sqrt(H) )   over graph neighbours only
H'_i     = sum_j  alpha_ij * (W h_j)
```

Instead of averaging all neighbours equally, the model **learns how much to weight each neighbour** and ignores unhelpful ones. On LA this matched the no-graph accuracy AND improved fairness (helped the poor regions).

5. Attacks by malicious clients (threat model)

A malicious police station can send a bad model update to poison the shared model. We implement three:

Attack	What the attacker does	Formula
Label-flip	Trains on flipped labels (crime<->no-crime)	Y -> 1 - Y
Scaling	Blows up its update to dominate the average	delta -> 8 * delta
Sparsity-camouflage (novel)	Pretends to be a quiet area to slip past defenses	Y -> 0 ; delta -> 0.3*delta

Key idea: The sparsity-camouflaged attack is our novel contribution: it hides inside the sparsity that the field is trying to handle, so defenses built to reject “odd” clients cannot tell it from an honest poor neighbourhood.

6. Defenses (server-side)

```
Trimmed-mean : drop the most-distant client updates, average the rest
Multi-Krum   : score_i = sum of distances to i's nearest updates; keep low scores
VOUCH (ours) : suspicion_i = Krum_score_i * density_i
               density_i = mean label-rate of client i's data
   -> sparse-but-honest clients (low density) are NOT penalised for looking odd;
      dense-yet-odd clients (real attackers) are flagged.
```

Multi-Krum and Bulyan reject outliers — but honest poor neighbourhoods ARE outliers, so they get wrongly removed. Our **vouch** rule divides the outlier score by the client's data density, so genuinely sparse clients are protected. (This is the piece that still needs the most tuning — see honest status.)

7. Fairness, week-ahead and uncertainty

7.1 Fairness metric

```
Regions split by total crime:  Head = top 20%, Mid = next 30%, Tail = bottom 50%
Fairness gap = MacroF1(Head) - MacroF1(Tail)     (smaller = fairer)
```

7.2 Week-ahead prediction

Change the label from “next day” to “the day H steps ahead” (H = 7 for a week). Harder, but useful for planning patrols.

```
X = days t .. t+L-1     ->     Y = day t + L + H - 1
```

7.3 Uncertainty (MC-dropout)

Keep dropout ON at test time, run the model T times, and report the spread of the predictions as the model's uncertainty.

```
uncertainty = std over T dropout runs of sigmoid(logit)
```

8. Results on the real data

8.1 Reproducing FedCrime (LA) — our numbers match the paper

Subset	Ours MacroF1	Paper
S-Alpha	71.70	72.16
S-Beta	65.18	61.70
S-Gamma	55.81	48.54
S-Omega	49.42	48.43

8.2 The over-smoothing fix (GNN vs No-GNN, clean, real data)

City	Model	Overall	Tail	Fair gap
LA	Plain GNN	48.09	27.51	39.14
LA	Gated GNN	53.94	21.21	48.03
LA	Attention GNN	53.21	28.04	41.28
LA	No-GNN	54.09	24.79	44.48
CHI	Plain GNN	55.95	28.21	47.06
CHI	Gated GNN	66.55	24.51	60.47
CHI	No-GNN	66.94	25.99	59.17

Key idea: Plain GNN loses 6–11 points (over-smoothing). The GATED graph recovers almost all of it (53.94 vs 54.09; 66.55 vs 66.94). On LA the ATTENTION graph matches accuracy AND is the fairest (Tail 28.04, gap 41.28). Reproducible on both cities.

8.3 Attacks vs defenses (LA, scaling attack, gated GNN)

Defense	Overall	Tail	Fair gap

FedAvg	0.00	0.00	0.00
Trimmed	49.63	9.93	57.41
Multi-Krum	50.82	10.16	57.94
Vouch (ours)	49.57	11.83	54.57

Under the scaling attack plain FedAvg collapses to 0; the robust defenses recover to ~50. Vouch is slightly fairer (lower gap) but not yet the clear winner.

9. Honest status — solved vs open

Goal	Status	Evidence
Reproduce FedCrime on real LA	SOLVED	Matches paper (72->49)
Add a GNN on real LA & Chicago	SOLVED	Runs on both cities
Fix GNN accuracy loss (over-smoothing)	SOLVED	Gated/attention recover accuracy
GNN improves fairness	SOLVED	Attention: higher Tail, lower gap
Show fairness problem is real	SOLVED	Head ~65 vs Tail ~25
Attacks break FedAvg	SOLVED	FedAvg -> 0 under attack
Week-ahead + uncertainty	SOLVED	Both run
A defense accurate AND fair everywhere	OPEN	Vouch not consistently best
Stable, repeatable results	OPEN	Numbers vary across runs
Publication-level rigour	OPEN	Needs seeds, stats, tuning

Honest note: The strongest, most reproducible result is the over-smoothing fix + attention fairness (Section 8.2). The attack/defense results are promising but not yet stable — turning “vouch” into a reliably winning defense is the main research work still ahead, best done with your professor (multiple seeds, averaging, tuning).

10. Recommended next steps

1. Build the paper around the solid result: **a gated/attention federated GNN that fixes over-smoothing and improves fairness for poor neighbourhoods under zero-inflation**.

2. Add the security angle as a second contribution: the **sparsity-camouflaged attack** and the **density-aware vouch defense** — after stabilising the numbers.

3. Run every experiment with multiple random seeds, report mean ± std, and add significance tests.

4. Target venue: IEEE TKDE, Information Fusion, IEEE TITS, or IEEE TIFS (for the security angle).

Reference: Bhumika, Lalanda, Vega, Das. FedCrime, Neurocomputing 679 (2026) 133217. Data: LA open-data portal; City of Chicago open-data portal.